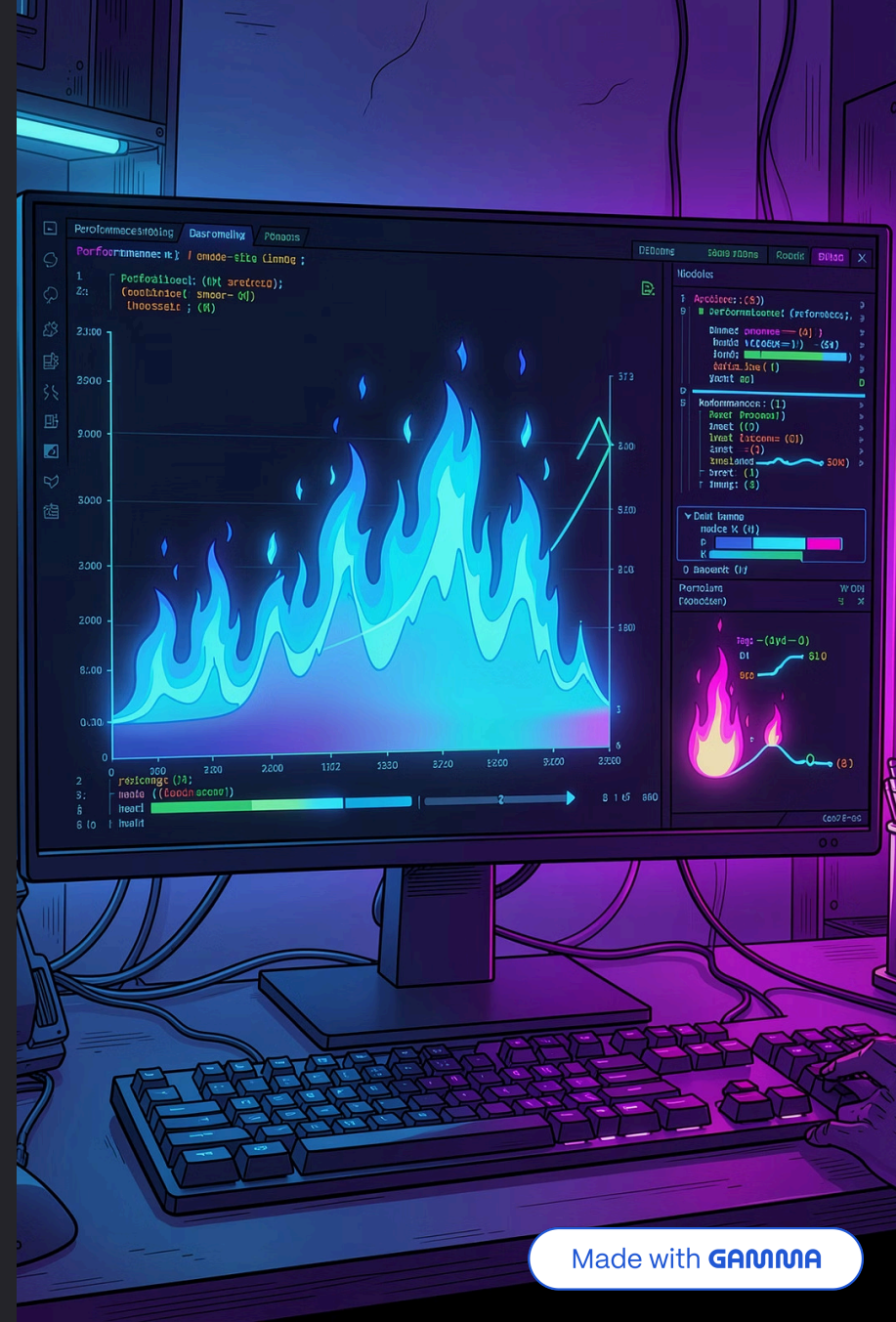


Flame Graph with AI Interpretation

Automatyczna diagnostyka wydajności z wykorzystaniem LLM

Franciszek Job · Dawid Zawiślak · Jakub Worek · Krzysztof Swędzioł

CEL: PREWENCJA AWARII POPRZEZ AI-POWERED INSIGHTS





Problem: Diagnostowanie wydajności to kosztowny bottleneck

Długi MTTR

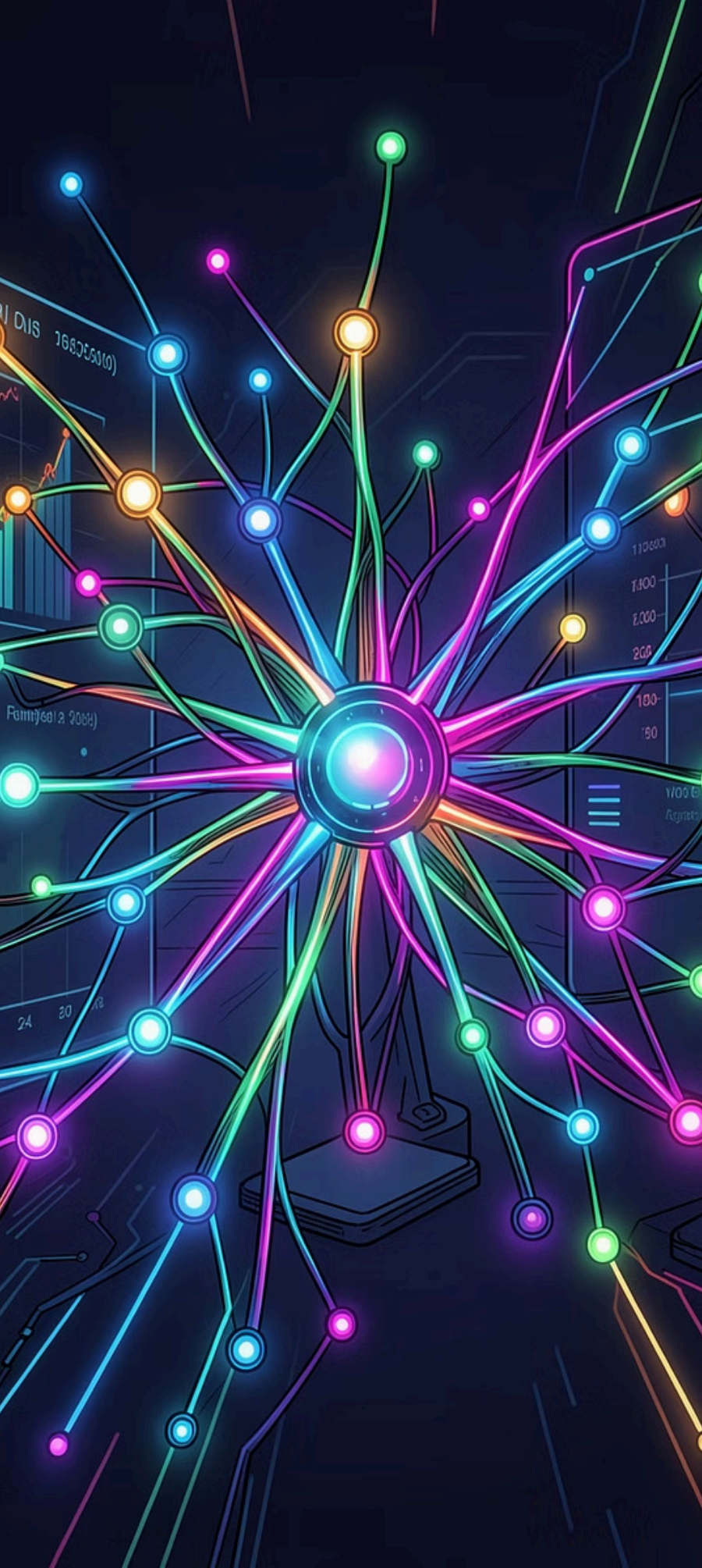
Godziny ręcznej analizy flame graphs przed zidentyfikowaniem root cause. Każdy incydent wydajnościowy to realna strata biznesowa.

Bariera wejścia

Interpretacja profili wymaga doświadczenia — Nie każdy od samego początku będzie w stanie rozpoznać odpowiednio problem.

Zero automatyzacji

Każda analiza bottlenecków wymaga zaangażowania eksperta. Brak skalowalności procesu diagnostycznego w zespołach.



Rozwiązanie: AI jako pierwszy responder wydajnościowy



Continuous Profiling

Zbieranie profili wydajności w czasie rzeczywistym — zero overhead w production, pełna widoczność każdej funkcji.



AI Interpretation

LLM analizuje flame graphs i generuje konkretne rekomendacje optymalizacyjne w naturalnym języku.



Model Context Protocol

Bezpieczny, standaryzowany most między infrastrukturą observability a modelem językowym.



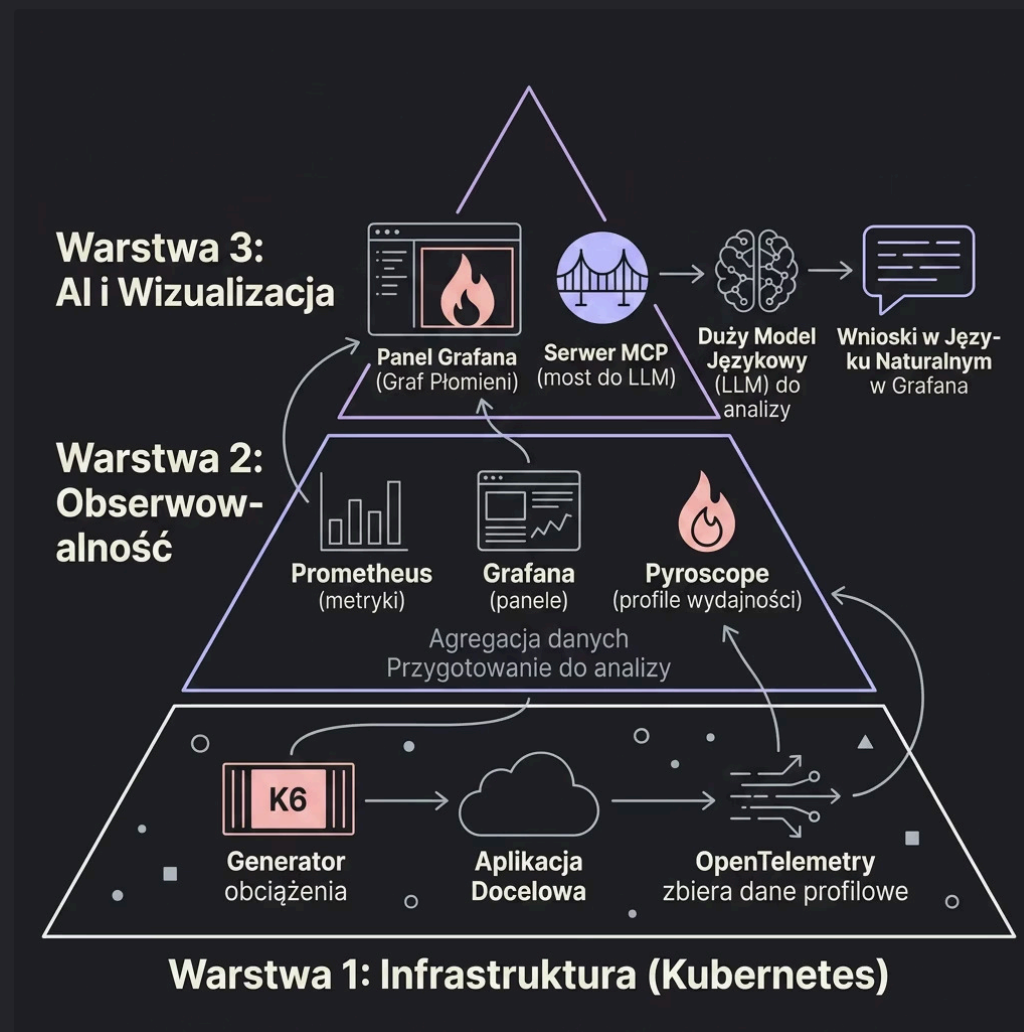
Automatyczna Diagnostyka

Natychmiastowe rekomendacje bezpośrednio w Grafana — bez opuszczania narzędzia developerskiego.

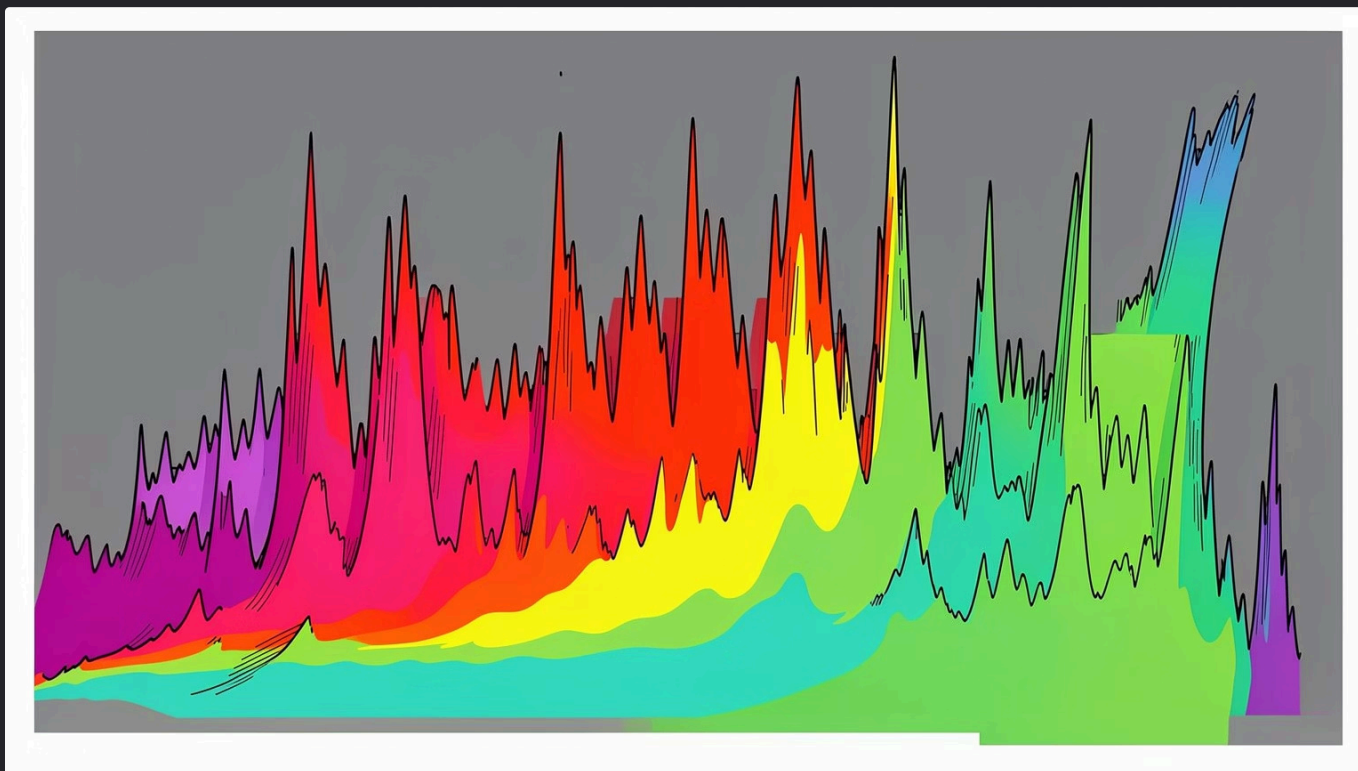
Architektura: 3 Warstwy Systemu

System Flame-AI składa się z trzech ściśle współpracujących warstw, które razem tworzą w pełni zautomatyzowany pipeline diagnostyczny — od generowania obciążenia po AI-powered rekomendacje.

Wynik końcowy to **Natural Language Insights** wyświetlane bezpośrednio w Grafana Dashboard.



Case Study: "Inefficient Sorter"



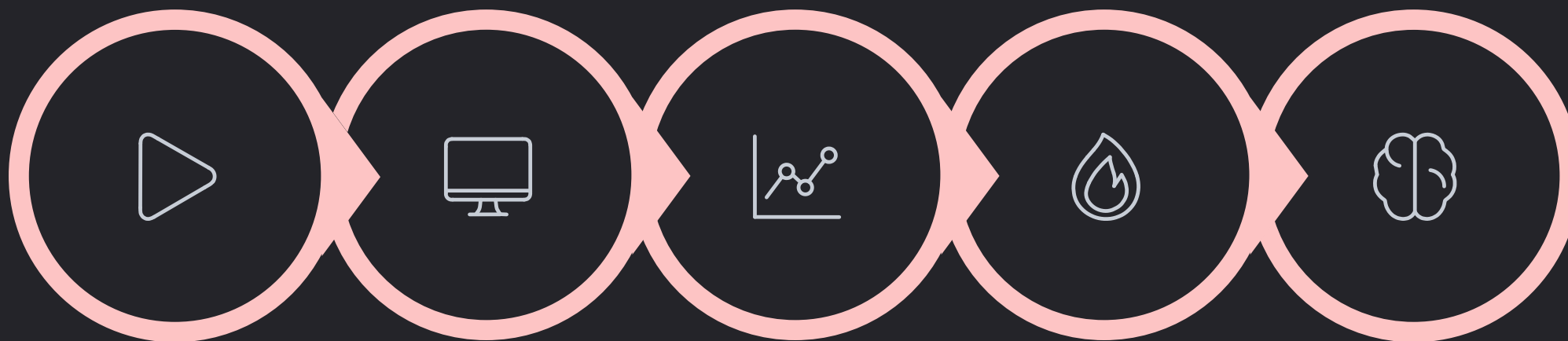
Aplikacja demonstracyjna

- **/fast** — Optymalne sortowanie $O(n \log n)$ via built-in `sorted()`
- **/slow** — Bubble Sort $O(n^2)$, celowo nieefektywny

Scenariusz testowy

1. K6 wysyła flood requestów na `/slow`
2. OpenTelemetry wykrywa 90% CPU w `bubble_sort()`
3. Pyroscope generuje Flame Graph z szerokim plateau
4. Developer klika **"Explain with AI"**
5. LLM odpowiada: *"92% czasu w bubble_sort. Zamień na QuickSort/MergeSort dla $O(n \log n)$ "*

Demo Flow



Uruchom

Generacja
obciążenia

Grafana

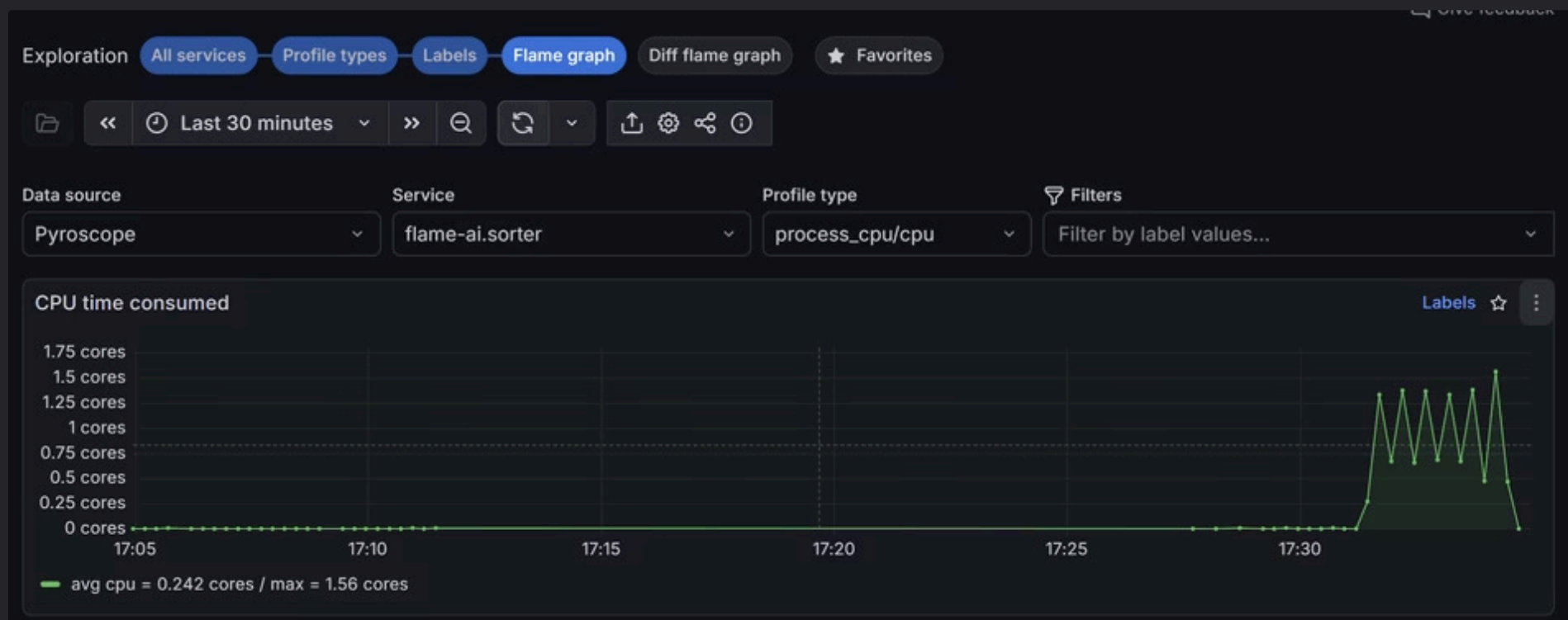
FlameGraph

Diagnoza

Cały pipeline od uruchomienia klastra po AI-powered diagnozę zajmuje **kilka minut**. Developer nie opuszcza środowiska Grafana.

Przykład działania

Wykres w Grafanie :



Przykład działania

Analiza z LLM :

Flame graph analysis

Performance Bottleneck

The primary performance bottleneck is the function `bubble_sort`, located in `sorters.py`. This single function is responsible for approximately 97.69% of the total CPU time, with the bulk of the execution spent within its own internal operations across lines 26, 27, 28, and 29. The application's main thread and worker threads are being almost entirely consumed by these recursive or repeated calls to the `bubble_sort` algorithm.

Root Cause

Stos Technologiczny

Komponent	Technologia	Rola
Orchestration	Kubernetes (minikube)	Zarządzanie kontenerami
Profiling	OpenTelemetry + Pyroscope	Zbieranie danych per funkcja
Load Testing	K6	Generowanie obciążenia
Visualization	Grafana Dashboard + Flame Graph	UI i interaktywna wizualizacja
AI Bridge	MCP Server	Bezpieczny protokół dla LLM
Brain	LLM	Analiza bottlenecków



Kubernetes



Pyroscope



Grafana



LLM + MCP

Implementacja Techniczna

Aplikacja — Python FastAPI

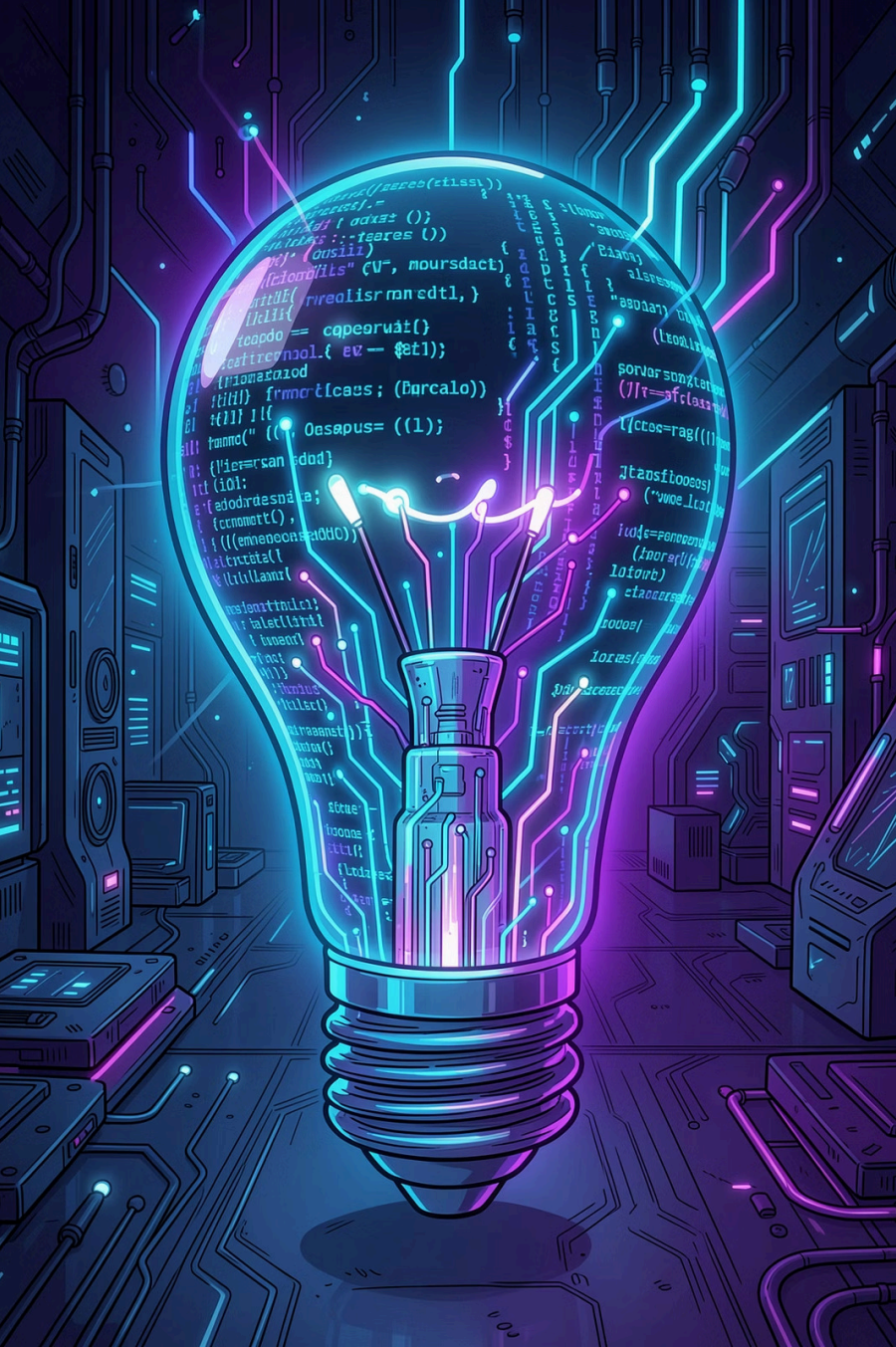
- 2 endpointy: `/fast` i `/slow`
- Integracja z Pyroscope profiler
- Deployment na Kubernetes (namespace: `flame-ai`)

Load Testing — K6

- Scenariusze: `slow-flood.js`, `mixed.js`
- Uruchomienie lokalnie lub in-cluster via k6-operator

Struktura projektu

```
flame-ai/  
├── app/  
│   ├── main.py # /fast, /slow  
│   └── sorters.py # algorytmy  
├── k8s/sorter/ # K8s manifests  
├── helm/ # Pyroscope + Grafana  
├── grafana/  
│   ├── dashboards/ # Flame graph JSON  
│   └── llm-prompts/ # AI prompt templates  
├── k6/scenarios/ # Load test scripts  
└── scripts/ # up.sh, down.sh
```



Kluczowe Innowacje

MCP Integration

Pierwszy praktyczny use case Model Context Protocol w observability

Real-time AI Analysis

Natychmiastowa interpretacja flame graphs bez ręcznej analizy

Developer Experience

Insights bezpośrednio w Grafana, zero kontekst-switching, zero dodatkowych narzędzi.

Korzyści Biznesowe i Podsumowanie

Znacznie szybsza diagnostyka

Z godzin ręcznej analizy do minut dzięki AI. Problemy wydajnościowe są identyfikowane i rozwiązywane natychmiast.

Automatyczna identyfikacja bottlenecków

System automatycznie wykrywa wąskie gardła w kodzie bez konieczności ręcznej analizy flame graphs.

Demokratyzacja wiedzy

Mniej doświadczone osoby mogą diagnozować problemy wydajności na zaawansowanym poziomie. Nie potrzeba głębokiego doświadczenia w profilowaniu.

Flame-AI to:

- Zautomatyzowana diagnostyka wydajności
- Analiza bottlenecków zasilana AI
- Profilowanie ciągłe w czasie rzeczywistym
- Przyjazny interfejs dla developerów

Kubernetes · Pyroscope · Grafana · MCP · LLM